A1.

The given schema violates all the three basic normalization (i.e. 1nf,2nf,3nf) and does not have a proper structure

The patient , doctor, medicine , amount are all listed in one table

One doctor can have multiple patients and one patient can go for multiple doctors for treatment.

One treatment procedure can have multiple medicines and the amount for each medicine is different.

We cannot define the relationship between the doctor , the patient and the medicine in one table without .

A single table cannot hold all the values. To have a proper structure and understanding the relationships between the entities , it is better to have multiple tables

1NF violation in the schema:

- Multiple patient phone numbers
- Prescription details
- Medicines
- Multiple payments per patient per appointment

2nf and 3nf violations:

- Schema is not in 1nf
- Doctor name,specialization has partial dependency on patient_id
- And other columns in the schema are also partial dependent

  patient_id -> doctor_name -> doctor_specialization
  patient_id->medicine_name->medicine_price
- This schema is very badly structured and is not normalized at all.
- The information about individual entity is tightly packed with others.hence its very difficult to change it in future
- There are no pk for individual entities to distinguish themselves

A2. Normalize the Schema

1nf:

Patients(
patient_id(pk),

```
        patient _name
)

patient_phones(
        phone_id(pk)
        patient_id(FK),
        phone_number
)

Patient_record(
        record_id (PK),
        patient_id (FK),
        medicine_id(fk),
        Doctor_name,
        Doctor_specialization,
        Appointment_date,
        Treatment,
        Bill_amount,
        payment_status

)

diagnosis(
         diagnosis_id(PK)
        record_id (FK),
         diagnosis_name
)
medicine(
        medicine_id (pk),
         Medicine_name,
         medicine_price

)
```

2nf:

```
Patients(
        patient_id(pk),
        patient _name
)
patient_phones(
```

```
        phone_id(pk),
        patient_id(FK),
        phone_number
)
Doctor(
         doctor_id (PK),
        doctor_name,
        Doctor_specialization(FK)

)

Specialization(
         Specialization_id(pk)
         Specialization_name
)
Medicine(
        medicine_id (PK),
        medicine_name,
        medicine_price
)

Appointment(
        appointment_id (PK),
        patient_id (FK),
        doctor_id (FK),
        appointment_date
)
Diagnosis(
        diagnosis_id (PK),
        diagnosis_name
)
Prescription_details(
        prescription_id (PK),
        appointment_id (FK)
)
treatments (
        treatement_id(pk),
        prescription_id(fk),
        medicine_id (FK),
        dosage,
        frequency
        )
```

Bill(
bill_id (PK),
appointment_id (FK),
bill_amount,
Payment_status
)

3nf:
Patients(
patient_id (PK),
name
)

Patient_Phones(
phone_id (PK),
patient_id (FK),
phone_number
)

Doctors(
doctor_id (PK),
name,
spec_id (FK),
dept_id (FK)
)
Specializations(
spec_id (PK),
name
)

Medicines(
medicine_id (PK),
name,
price
)

Appointments(
appointment_id (PK),
patient_id (FK),
doctor_id (FK),

appointment_date,
appointment_time
)

Diagnoses(
diagnosis_id (PK),
appointment_id (FK),
diagnosis_description
)

Prescription_Details(
prescription_id (PK),
appointment_id (FK)
)

Treatments(
treatment_id (PK),
prescription_id (FK),
medicine_id (FK),
dosage,
frequency,
whenToTake
)

Bills(
bill_id (PK),
bill_amount,
appointment_id (FK)
)

Payments(
payment_id (PK),
bill_id (FK),
payment_date,
payment_status
)

Sql quries:

```
create table if not exists patients (
```

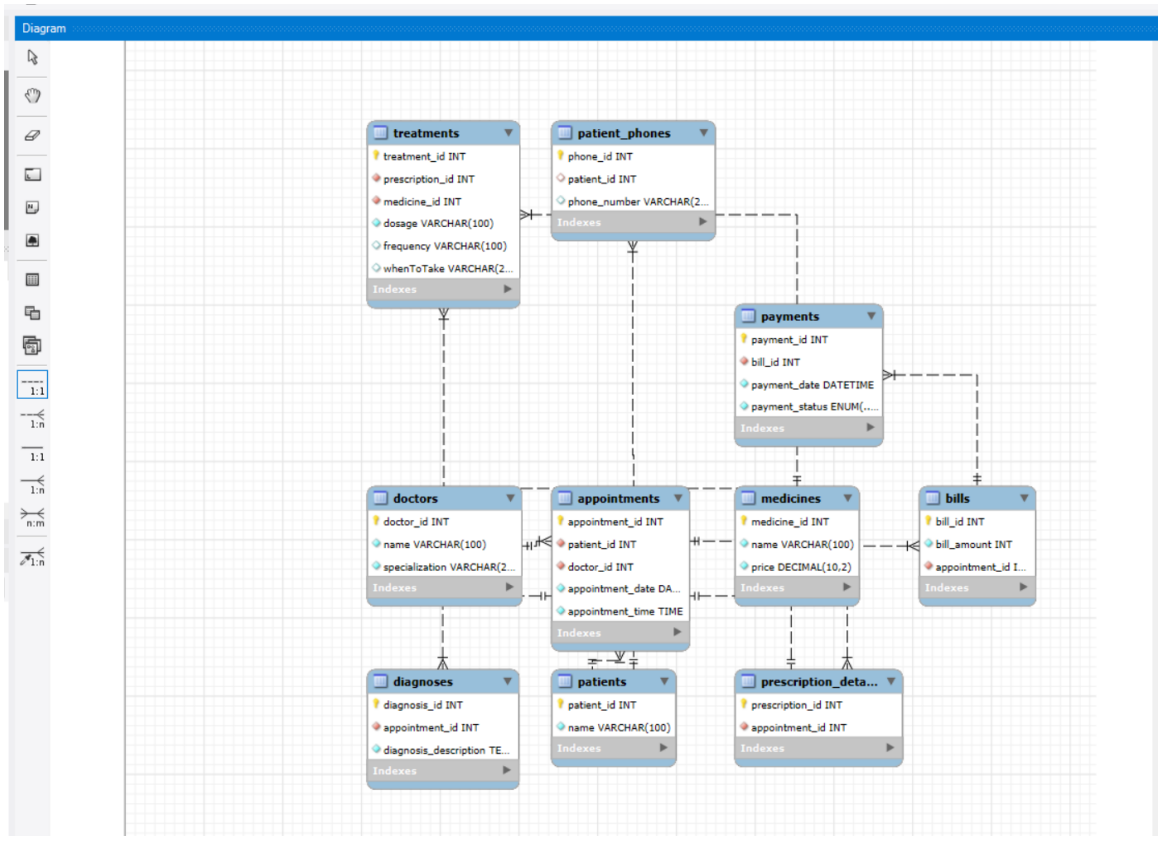
```
    patient_id INT auto_increment primary KEY,
    name varchar(100) not null
    );

CREATE TABLE if not exists patient_phones (
    phone_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT not null,
    phone_number VARCHAR(20) not null,
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
        ON DELETE CASCADE
) ;
create table if not exists specializations(
    spec_id int auto_increment primary key,
    name varchar(20) not null
    );

create table if not exists doctors (
    doctor_id INT auto_increment primary KEY,
    name varchar(100)  not null,
    spec_id int not null,
    foreign key(spec_id) references specializations(spec_id) ON DELETE CASCADE,

    );

create table if not exists medicines (
    medicine_id INT auto_increment primary KEY,
    name varchar(100)  not null,
    price DECIMAL(10,2) not null
    );
create table if not exists appointments(
    appointment_id int auto_increment primary key,
    patient_id INT Not null,
    doctor_id INT Not null,
    appointment_date date not null,
    appointment_time time not null,
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
      on delete cascade,
            foreign key (doctor_id) REFERENCES doctors(doctor_id)
      on delete cascade
    );
```

```
create table if not exists diagnoses(
      diagnosis_id int auto_increment primary key,
      appointment_id int not null,
      diagnosis_description TEXT ,
      FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
      on delete cascade);



CREATE TABLE prescription_details (
   prescription_id INT AUTO_INCREMENT PRIMARY KEY,
   appointment_id INT NOT NULL,
   UNIQUE (appointment_id),
   FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id) ON
DELETE CASCADE
);
CREATE TABLE treatments (
   treatment_id INT AUTO_INCREMENT PRIMARY KEY,
   prescription_id INT NOT NULL,
   medicine_id INT NOT NULL,
   dosage VARCHAR(100) NOT NULL,
   frequency VARCHAR(100),
   whenToTake VARCHAR(20),

   FOREIGN KEY (prescription_id)
      REFERENCES prescription_details(prescription_id)
      ON DELETE CASCADE,

   FOREIGN KEY (medicine_id)
      REFERENCES medicines(medicine_id)
      ON DELETE CASCADE
);
create table if not exists bills(
   bill_id int auto_increment primary key,
   bill_amount DECIMAL(10,2) not null,

   appointment_id int not null,
   FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
     on delete cascade
   );
```

```
CREATE TABLE if not exists payments (
   payment_id INT AUTO_INCREMENT PRIMARY KEY,
   bill_id INT NOT NULL,
   payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
   payment_status ENUM('pending', 'completed', 'failed') NOT NULL default
'pending',
   FOREIGN KEY (bill_id)
      REFERENCES bills(bill_id)
      ON DELETE CASCADE
);
```

A3. Assumptions

1.One patient can have multiple phones
2.one doctor has only one specialisation and multiple doctors can have same specialization
3.Same patient multiple appointments
4.One prescription per treatment
5.One patient can book many appointments and One doctor can handle many appointments
6. One appointment can have multiple diagnoses and one prescription per diagnosis
6.Billing->per treatment per patient
7.each bill can have multiple payment attempts and the can have payment status
8.Each treatment can have multiple medicines
9.Each medicine has its own price



B1.

1. can a doctor be part of multiple departments and can he have multiple specislization
2. Should there be a minimum time duration between appointments and can a patient have multiple appointments in a single day
3. Is there any need to track the appointment status and if so how effectively is it integrated into the system for maximum efficiency
4. will there be multiple consultation records from different doctors schedules for a single appointment
5. Is I task of the system to track the doctor's timings/schedule also
6. When does the system delete the records of a patient
   * after a certain period of time ??
   * or is it done manually

B2 .Updated schema

Patients(
  patient_id (PK)
)

```
Patient_Phones(
   phone_id (PK),
   patient_id (FK)
)

Departments(
   dept_id (PK)
)

Specializations(
   spec_id (PK)
)

Doctors(
   doctor_id (PK),
   spec_id (FK),
   dept_id (FK)
)

Medicines(
   medicine_id (PK)
)

Appointments(
   appointment_id (PK),
   patient_id (FK),
   doctor_id (FK),
   follow_up_to (FK)
)

Consultations(
   consultation_id (PK),
   appointment_id (FK),
   doctor_id (FK)
)

Diagnoses(
   diagnosis_id (PK),
   consultation_id (FK)
)
```

Prescription_Details(
prescription_id (PK),
consultation_id (FK)
)

Treatments(
treatment_id (PK),
prescription_id (FK),
medicine_id (FK)
)

Bills(
bill_id (PK),
consultation_id (FK)
)

Payments(
payment_id (PK),
bill_id (FK)
)

B3.

For the changes during development part, I just added the dockor_id as foreign key in a table called consultation.And I made the consultation_id to be the main foreign attribute for the other tables like prescription details,bills etc…

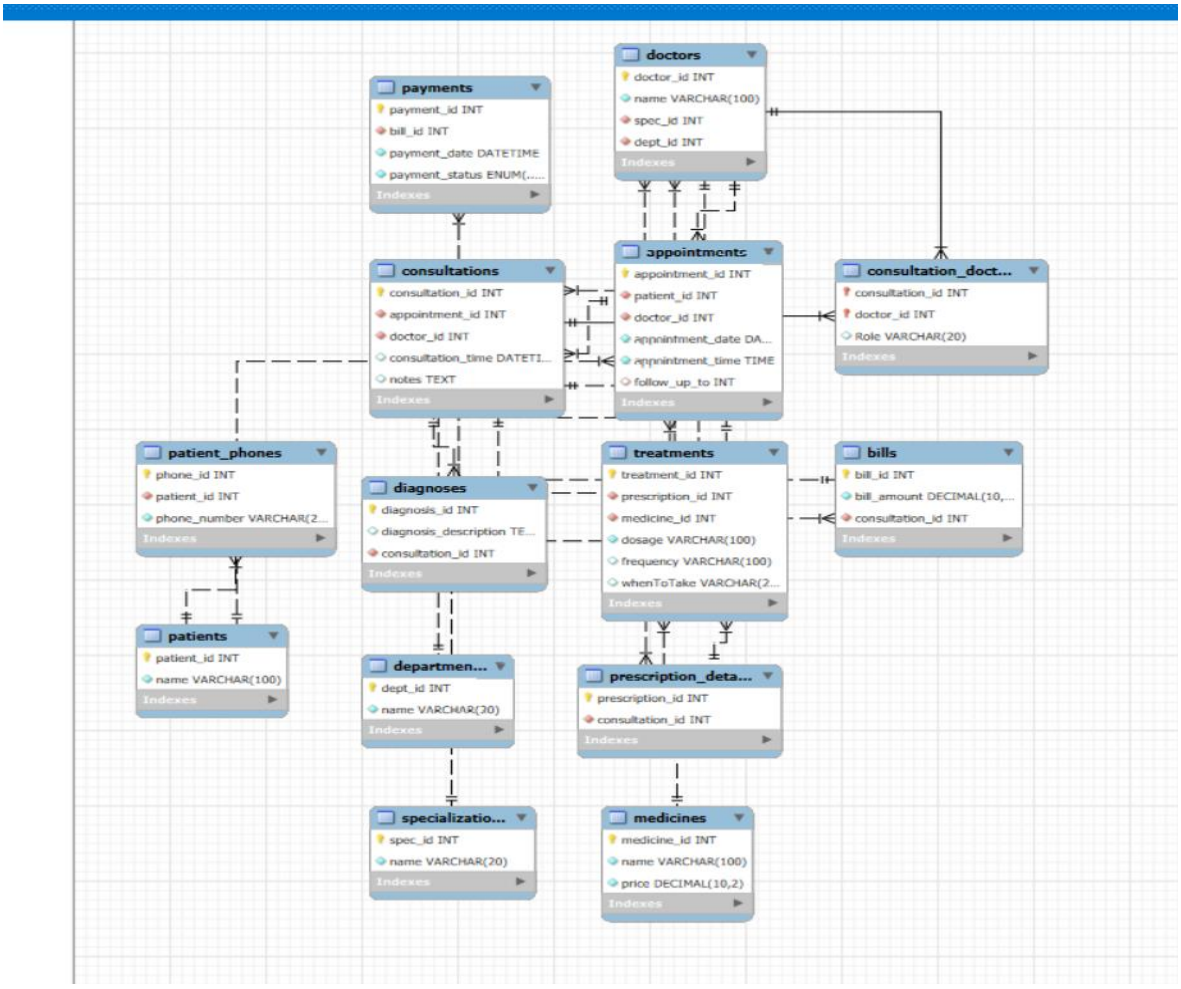
For storing the doctor's role , another table called consultation_doctors

Consultations(

consultation_id (PK),
appointment_id (FK),
doctor_id (FK)
)

Consultation_Doctors(

consultation_id PK,
doctor_id PK,
Role,
consultation_id FK,
doctor_id FK);



Section C:

1.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

patient_id	patient_name	consultation_id	consultation_time	doctor_name	diagnosis_description	medicine_name	dosage	frequency	whenToTake
1	Ravi Kumar	9	2026-04-29 10:02:23	Dr. Vivek Menon	Improving Angina	Aspirin	75mg	Once daily	After food
1	Ravi Kumar	10	2026-04-29 10:02:23	Dr. Ramesh Gupta	Mild Fatigue	Paracetamol	500mg	Once daily	After food
1	Ravi Kumar	1	2026-04-29 08:39:37	Dr. Vivek Menon	Mild Angina	Aspirin	75mg	Once daily	After food
1	Ravi Kumar	8	2026-04-29 08:39:37	Dr. Vivek Menon	Stable Condition	Aspirin	75mg	Once daily	After food

2.

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

doctor_id	name	total_consultations
1	Dr. Vivek Menon	3

3.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

patient_id	name	total_outstanding
1	Ravi Kumar	750.00
3	Mohammed Arif	600.00
5	Karthik Rao	350.00

4.